

Part 3 — instant-ngp Analysis

Emre Can Kaya

A. Where Does the Rendering Step Take Place?

The rendering pipeline in instant-ngp lives inside two CUDA source files:

- **src/testbed_nerf.cu** — contains the core NeRF rendering kernels: ray generation, occupancy-grid-based ray marching, network evaluation, and front-to-back alpha compositing.
- **src/testbed.cu** — contains the interactive testbed loop (GUI, training dispatch, snapshot I/O) and calls into testbed_nerf.cu each frame.

Supporting headers pulled in by testbed_nerf.cu:

- **nerf_network.h** — multiresolution hash encoding + tiny MLP definition.
- **render_buffer.h** — GPU framebuffer management.
- **marching_cubes.h** — mesh extraction (off the rendering hot path, used for export).

All heavy computation runs as fully-fused CUDA kernels inside the tiny-cuda-nn framework. "Fully-fused" means the entire MLP forward pass executes in a single kernel without round-trips to global memory between layers. This is the primary reason instant-ngp can render at tens of milliseconds per 1920×1080 frame on an RTX 3090.

B. What Algorithms Infer Missing Surfaces?

Instant-ngp does not complete geometry in the traditional sense. It learns a continuous 5D function (3D position + 2D view direction) $\rightarrow$ (volume density σ , view-dependent radiance c). What we perceive as a surface is simply the region in space where σ transitions sharply from near-zero to high and there is no explicit mesh during rendering. The model has no concept of "surface" or "missing region"; it assigns density and color to every point in the bounding volume.

With that said, the mechanisms that allow the model to produce plausible-looking geometry in unobserved or sparsely observed regions are:

1. Multi-Resolution Hash Encoding (the headline contribution)

Rather than feeding raw coordinates to the MLP, each 3D point is encoded by looking up L levels of voxel grids at geometrically-spaced resolutions (typically $L = 16$, from $\sim 16^3$ to $\sim 512^3$). Each grid stores F -dimensional feature vectors ($F = 2$ in the paper) in a fixed-size hash table of T slots.

Three properties make this work for implicit surface reconstruction:

- Trilinear interpolation between the 8 surrounding voxel features at each resolution level produces a smooth, differentiable encoding so that the gradients flow cleanly and nearby positions share information.
- Concatenating L levels gives the downstream MLP a feature that is simultaneously coarse (global structure) and fine (local detail). Coarse levels generalise across the scene; fine levels specialise where training views are dense.
- Hash collisions at fine resolutions are tolerated because the multi-level structure almost always disambiguates them: a collision at level k is resolved by levels $k-1$ or $k+1$, which would probably hash the same position differently.

The practical effect: the MLP can be shrunk to 2–4 hidden layers of 64 neurons (versus 8 layers of 256 in vanilla NeRF), because the encoding itself carries the high-frequency capacity. A smaller network generalizes better from sparse observations.

2. Occupancy-Grid-Accelerated Ray Marching

A separate multi-scale occupancy grid stores one bit per voxel: occupied or empty. Candidate samples whose occupancy falls below a threshold are skipped; samples behind regions where accumulated transmittance has already fallen below $\sim 10^{-4}$ are also skipped (early ray termination).

The grid is maintained alongside the network and periodically resampled from the network's own density predictions with an exponential moving-average decay. This creates a feedback loop: as training progresses and the density field sharpens, the occupancy grid becomes a tighter mask, which reduces the sample count per ray, which in turn makes each training iteration faster. The paper reports 10–100× speedup over naive uniform sampling.

From a geometry-inference perspective, the occupancy grid acts as a soft prior: once the network decides a region is empty, that region receives no further gradient signal from ray marching. This accelerates convergence but can also freeze regions that are hard to observe which is a known limitation.

3. View-Dependent Appearance

Volume density σ is a function of position only. Radiance c is a function of both position and view direction and the direction is encoded with a 4-band spherical harmonics expansion (16 coefficients) and concatenated with the geometry MLP output before the colour MLP. This allows the model to represent specular highlights and inter-reflections that would otherwise show up as blurry density artefacts.

C. How Is View Synthesis Computed?

For each pixel of a requested novel view, the pipeline executes the following steps:

Step 1 — Ray construction

A camera ray $r(t) = o + t \cdot d$ is constructed from the camera intrinsics K (which map a pixel to a viewing direction) and the extrinsic pose (camera rotation and position in world space). The ray origin o is the camera center; the direction d is obtained by unprojecting the pixel through K and rotating it by R into world space. This ray is intersected with the scene's axis-aligned bounding box (AABB) to obtain the entry and exit distances t_{near} and t_{far} , which bound the segment where sampling takes place.

Step 2 — Occupancy-guided marching

The ray is marched from t_{near} to t_{far} with an exponentially growing step size (controlled by `cone_angle_constant` in the testbed). At each candidate sample, the occupancy grid is queried; if the cell is marked empty, the sample is skipped and the ray jumps to the next occupied cell. This replaces the original NeRF's fixed uniform sampling and is the main reason rendering is near real-time capable.

Step 3 — Network evaluation

Surviving samples are batched and evaluated by the network in two sub-passes:

- Position $\rightarrow$ hash-grid encoding $\rightarrow$ density MLP $\rightarrow (\sigma, \text{geometry feature vector})$.
- (geometry feature, SH-encoded view direction) $\rightarrow$ colour MLP $\rightarrow$ RGB.

Both MLPs run inside a single fully-fused CUDA kernel.

Step 4 — Volume rendering integral

Samples are composited front-to-back using the standard discrete emission-absorption integral:

$$C(r) = \sum_{i=1}^N T_i (1 - e^{-\sigma_i \delta_i})^{i-1} c_i, \quad T_i = \exp \left(- \sum_{j=1}^{i-1} \sigma_j \delta_j \right)$$

where δ_i is the distance between consecutive samples. Transmittance T_i is accumulated front-to-back; once it drops below $\sim 10^{-4}$ the ray is terminated early. There is no triangle rasterization, no depth buffer, no view blending and the pixel colour is entirely determined by the learned volumetric field along that ray.

Key distinction from image-based rendering

Novel-view synthesis here is not interpolation between stored images. It is a volumetric render of a continuous function that was trained to be photometrically consistent across all input cameras. The

model generalizes to new viewpoints because the 5D radiance field is smooth and the multi-resolution encoding captures structure at multiple scales.

References

Müller, T., Evans, A., Schied, C., Keller, A. *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding*. ACM TOG (SIGGRAPH) 2022. arXiv:2201.05989.

NVlabs/instant-ngp source — rendering kernels: `src/testbed_nerf.cu`. github.com/NVlabs/instant-ngp

Mildenhall, B. et al. *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. ECCV 2020. (Volume rendering integral and 5D field formulation.)